

PyDiag

Challenge Pratique Python

Gestion des Exceptions & Manipulation de Fichiers

Notions évaluées

Exceptions, try/except/else/finally, exceptions personnalisées, lecture/écriture de fichiers, context manager `with`, combinaison exceptions/fichiers

Table des matières

Contexte	2
Mission de l'apprenant	2
1 Bloc 1 — Notion d'exception & bloc try/except (30 min — 20 pts)	2
2 Bloc 2 — Levée d'exceptions (30 min — 20 pts)	4
3 Bloc 3 — Manipulation de fichiers (30 min — 25 pts)	5
4 Bloc 4 — Croisement exceptions / fichiers (30 min — 35 pts)	6
5 Règles de Gestion	7
6 Jeu de données de test (exemple global)	8
7 Structure attendue du script	8
8 Livrables attendus	9

Contexte

Ce challenge évalue la maîtrise de la **gestion des erreurs** et de la **manipulation de fichiers** en Python. Il est organisé en 4 blocs progressifs : les fondamentaux des exceptions, la levée d'exceptions (y compris personnalisées), la manipulation de fichiers (lecture/écriture/context manager), puis un dernier bloc qui combine exceptions et fichiers dans des cas proches de situations réelles (journal de commandes, fichiers de notes, import CSV/JSON).

L'objectif est d'évaluer la capacité de l'apprenant à :

- Différencier une erreur de syntaxe d'une exception (erreur d'exécution)
- Intercepter et traiter les exceptions de façon ciblée avec `try/except/else/finally`
- Lever des exceptions avec `raise`, y compris des exceptions personnalisées
- Lire et écrire des fichiers de façon sécurisée, en utilisant `with`
- Combiner gestion des erreurs et accès fichiers dans un scénario réaliste

Mission de l'apprenant

Pour chaque exercice, vous devez écrire une fonction ou un script Python autonome, testé avec le jeu de données fourni dans l'encadré « Exemple » correspondant. Respectez les bonnes pratiques : nommage clair (snake_case), fonctions courtes à responsabilité unique, gestion ciblée des exceptions (jamais de `except nu`), et systématiquement `with open(...)` pour les fichiers.

1 Bloc 1 — Notion d'exception & bloc try/except (30 min — 20 pts)

Bloc 1 : Fondamentaux des exceptions

- ☐ Différencier une erreur de syntaxe d'une exception (runtime)
- ☐ Situer une exception dans la hiérarchie (`BaseException`, `Exception`)
- ☐ Écrire un bloc `try/except` basique
- ☐ Intercepter une exception spécifique (`ZeroDivisionError`, `ValueError`...)
- ☐ Intercepter plusieurs types d'exceptions dans un même bloc
- ☐ Utiliser `else` et `finally` à bon escient

Exemple — Données & résultat attendu

1.1 — Erreur de syntaxe ou exception ?

```
extrait_a = "print('bonjour')"  
extrait_b = "resultat = 10 / 0"  
extrait_c = "valeurs = [1, 2, 3]\nprint(valeurs[5])"
```

Résultat attendu : pour chaque extrait, indiquer s'il s'agit d'une `SyntaxError` (détectée avant l'exécution) ou d'une exception levée à l'exécution, et préciser laquelle.

```
extrait_a -> SyntaxError (parenthese non fermee)  
extrait_b -> ZeroDivisionError (exception a l'execution)  
extrait_c -> IndexError (exception a l'execution)
```

Exemple — Données & résultat attendu**1.2 — division_securisee(a, b)**

```
division_securisee(10, 2)
division_securisee(10, 0)
```

Résultat attendu :

```
5.0
Erreur : division par zero impossible.
```

Exemple — Données & résultat attendu**1.3 — convertir_entier(valeur)**

```
convertir_entier("42")
convertir_entier("abc")
```

Résultat attendu :

```
42
Erreur : "abc" n'est pas un entier valide.
```

Exemple — Données & résultat attendu**1.4 — acceder_element(liste, index)**

```
notes = [12, 15, 9]
acceder_element(notes, 1)
acceder_element(notes, 10)
```

Résultat attendu :

```
15
Erreur : index 10 hors limites (taille de la liste : 3).
```

Exemple — Données & résultat attendu**1.5 — acceder_cle(dictionnaire, cle)**

```
eleve = {"nom": "Sara", "age": 20}
acceder_cle(eleve, "nom")
acceder_cle(eleve, "email")
```

Résultat attendu :

```
Sara
Erreur : la cle "email" n'existe pas.
```

Exemple — Données & résultat attendu**1.6 — try / except / else / finally**

```
traiter_valeur("8")
traiter_valeur("x")
```

Sortie attendue :

```
Conversion reussie : 8
Traitement termine.
Erreur : "x" n'est pas convertible.
Traitement termine.
```

2 Bloc 2 — Levée d'exceptions (30 min — 20 pts)

Bloc 2 : raise & exceptions personnalisées

- ☐ Lever une exception avec **raise**
- ☐ Relancer une exception interceptée (**raise** sans argument)
- ☐ Créer une exception personnalisée par héritage de **Exception**
- ☐ Utiliser une exception personnalisée dans une fonction métier

Exemple — Données & résultat attendu**2.1 — verifier_age(age) avec raise**

```
verifier_age(25)
verifier_age(-3)
```

Résultat attendu :

```
Age valide : 25
ValueError: l'age ne peut pas etre negatif (-3).
```

Exemple — Données & résultat attendu**2.2 — Relancer une exception (raise sans argument)**

```
traiter_liste_de_valeurs(["3", "9", "x", "5"])
```

Comportement attendu : la fonction journalise un message contenant la valeur fautive ("x") puis relance l'exception d'origine sans la modifier, afin que l'appelant en soit informé.

```
Log : valeur "x" invalide, exception relancee.
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: 'x'
```

Exemple — Données & résultat attendu**2.3 — Exception personnalisée StockInsuffisantError**

```
class StockInsuffisantError(Exception):  
    pass
```

Comportement attendu : l'exception doit accepter un message clair précisant le produit et la quantité manquante, par exemple :

```
StockInsuffisantError: stock insuffisant pour "bananes"  
(demande : 10, disponible : 4)
```

Exemple — Données & résultat attendu**2.4 — retirer_stock(stock, produit, quantite)**

```
stock = {"pommes": 20, "bananes": 4}  
retirer_stock(stock, "pommes", 5)  
retirer_stock(stock, "bananes", 10)
```

Résultat attendu :

```
Retrait effectue : 5 pommes.  
StockInsuffisantError: stock insuffisant pour "bananes"  
(demande : 10, disponible : 4)
```

3 Bloc 3 — Manipulation de fichiers (30 min — 25 pts)

Bloc 3 : Lecture, écriture & with

- ☐ Ouvrir un fichier avec `open()` et connaître les modes (`r`, `w`, `a`, `x`, `b`, `+`)
- ☐ Écrire dans un fichier avec `write()` / `writelines()`
- ☐ Différencier le mode `w` (écrasement) du mode `a` (ajout)
- ☐ Lire un fichier avec `read()`, `readline()`, `readlines()`
- ☐ Parcourir un fichier ligne par ligne avec une boucle
- ☐ Utiliser le context manager `with` pour une fermeture automatique

Exemple — Données & résultat attendu**3.1 — ecrire_liste_courses(chemin, articles) (mode w)**

```
articles = ["pommes", "lait", "pain"]  
ecrire_liste_courses("courses.txt", articles)
```

Contenu attendu de courses.txt :

```
pommes  
lait  
pain
```

Exemple — Données & résultat attendu**3.2 — ajouter_article(chemin, article) (mode a)**

```
ajouter_article("courses.txt", "oeufs")
```

Contenu attendu de courses.txt après l'ajout :

```
pommes
lait
pain
oeufs
```

Exemple — Données & résultat attendu**3.3 — lire_fichier(chemin) avec with et readlines()**

```
lire_fichier("courses.txt")
```

Résultat attendu :

```
['pommes\n', 'lait\n', 'pain\n', 'oeufs']
```

Exemple — Données & résultat attendu**3.4 — compter_lignes(chemin) avec parcours ligne par ligne**

```
compter_lignes("courses.txt")
```

Résultat attendu :

```
Nombre de lignes : 4
```

Exemple — Données & résultat attendu**3.5 — Modes d'ouverture**

```
modes_a_identifier = ["r", "w", "a", "x", "rb", "r+"]
```

Résultat attendu : pour chaque mode, préciser son effet (lecture seule, écrasement, ajout, création exclusive, binaire, lecture/écriture) et le comportement si le fichier n'existe pas.

4 Bloc 4 — Croisement exceptions / fichiers (30 min — 35 pts)

Bloc 4 : Combinaison des deux notions

- ☐ Combiner `try/except` avec l'ouverture d'un fichier
- ☐ Gérer `FileNotFoundError` et `PermissionError` lors d'un accès fichier
- ☐ Utiliser `finally` pour garantir la fermeture d'un fichier
- ☐ Lire un fichier CSV et gérer les lignes mal formées
- ☐ Lire/écrire un fichier JSON avec gestion d'erreurs
- ☐ Mini-challenge final combinant fichiers, exceptions et structures de données

Exemple — Données & résultat attendu**4.1 — lire_fichier_securise(chemin)**

```
lire_fichier_securise("courses.txt")
lire_fichier_securise("inexistant.txt")
```

Résultat attendu :

Contenu du fichier renvoyé sous forme de liste de lignes.
Erreur : le fichier "inexistant.txt" n'existe pas.

Exemple — Données & résultat attendu**4.2 — Import CSV : calculer_moyenne_csv(chemin)**

```
# notes.csv
nom,note
Ali,14
Sara,18
Lina,abc
Karim,9
```

Comportement attendu : la ligne Lina,abc doit déclencher une exception interceptée (note ignorée avec message), sans interrompre le traitement des autres lignes.

Attention : note invalide pour "Lina" ("abc"), ligne ignorée.
Moyenne calculée (3 notes valides) : 13.67

Exemple — Données & résultat attendu**4.3 — Mini-challenge final : journal de commandes**

```
stock = {"pommes": 20, "bananes": 4, "oranges": 15}

commandes_brutes = [
    "pommes,5",
    "bananes,10",
    "kiwis,2",
    "oranges,abc",
    "oranges,5",
]
```

Comportement attendu : pour chaque commande, tenter de retirer la quantité du stock (réutiliser `StockInsuffisantError` et une `KeyError` pour un produit inconnu), écrire chaque résultat (succès ou erreur) dans un fichier `journal.txt` via `with`, puis afficher le contenu final du journal.

```
[OK] pommes : -5 (reste 15)
[ERREUR] bananes : stock insuffisant (demande 10, dispo 4)
[ERREUR] kiwis : produit inconnu
[ERREUR] oranges : quantite invalide ("abc")
[OK] oranges : -5 (reste 10)
```

5 Règles de Gestion

Exceptions

- Une exception ne doit jamais être interceptée par un `except` nu ; toujours cibler le type attendu
- Une exception personnalisée doit hériter de `Exception` (jamais directement de `BaseException`)
- Un `raise` sans argument ne peut être utilisé qu'à l'intérieur d'un bloc `except`, pour relancer l'exception en cours

Fichiers

- Tout accès fichier doit se faire via `with open(...)`, jamais `open()/close()` manuels
- L'encodage doit toujours être précisé explicitement (`encoding="utf-8"`)
- Le mode `w` écrase le contenu existant ; le mode `a` l'ajoute à la suite ; ce choix doit être justifié dans le code

Combinaison

- Un accès fichier doit systématiquement gérer au minimum `FileNotFoundError`
- `finally` doit être utilisé uniquement quand une action doit être garantie, même en cas d'erreur (ex. fermeture, journalisation)
- Une donnée invalide dans un fichier ne doit jamais interrompre le traitement des autres données valides

6 Jeu de données de test (exemple global)

Jeu de données combiné, prêt à copier-coller, pour tester l'ensemble des blocs sur un même scénario.

Données d'entrée

```
stock_test = {"pommes": 20, "bananes": 4, "oranges": 15}

commandes_brutes_test = [
    "pommes,5",
    "bananes,10",
    "kiwis,2",
    "oranges,abc",
    "oranges,5",
]

notes_csv_test = """nom,note
Ali,14
Sara,18
Lina,abc
Karim,9
"""

fichier_test = "courses.txt"
fichier_inexistant_test = "inexistant.txt"
```

7 Structure attendue du script

```
challenge_exceptions_fichiers.py
|
|-- division_securisee(a, b)
|-- convertir_entier(valeur)
|-- acceder_element(liste, index)
|-- acceder_cle(dictionnaire, cle)
|-- verifier_age(age)
|-- StockInsuffisantError(Exception)
|-- retirer_stock(stock, produit, quantite)
|-- ecrire_liste_courses(chemin, articles)
|-- ajouter_article(chemin, article)
|-- lire_fichier(chemin)
|-- compter_lignes(chemin)
|-- lire_fichier_securise(chemin)
|-- calculer_moyenne_csv(chemin) # bloc 4
|-- traiter_journal_commandes(stock, commandes_brutes, chemin_journal) # bloc 4
```

8 Livrables attendus

1. Un script `.py` unique, fonctionnel de bout en bout pour les 4 blocs
2. Chaque fonction testée avec au moins le jeu de données fourni dans son encadré « Exemple »
3. Une exécution finale démontrant le mini-challenge du bloc 4 (4.3), y compris le fichier `journal.txt` généré